

HDOC DAY-41

Title: Advanced Paragraph String Processing & Frequency Analysis using Call-by-Reference Pointers

1. Problem Statement

Write a menu-driven C program that reads two paragraphs and uses separate user-defined functions for each operation with call-by-reference pointers:

1. Read two paragraphs from the user.

```
void readParagraph(char paragraph[], size_t size);
```

2. For each paragraph, identify and display the longest and shortest words.

```
void findLongestAndShortestWords(const char  
paragraph[], char longestWord[], char shortestWord[]);
```

3. Remove repeated consecutive spaces from each paragraph and display the modified paragraph.

```
void removeRepeatedSpaces(const char paragraph[], char  
modifiedParagraph[]);
```

4. Convert each paragraph to title case and display the modified paragraph.

```
void convertToTitleCase(const char paragraph[], char  
titleCaseParagraph[]);
```

5. Find and display the frequency of every distinct word in each paragraph (case-insensitive).

```
void findWordFrequencies(const char paragraph[]);
```

6. Exit.

2. Header Files Used

- <stdio.h>: Input and output stream operations (fgets, printf, scanf).
- <string.h>: Tokenization and string manipulations (strtok, strlen, strcpy, strcmp).
- <ctype.h>: Character classification utilities (isspace, isalpha, tolower, toupper).
- <stdlib.h>: Memory management functions.

3. Test Case Table

Test Case	Task	Input Sample	Expected Output	Status
1	Longest & Shortest	"C programming is fun"	Longest: programming	Pass

Test Case	Task	Input Sample	Expected Output	Status
			Shortest: C	
2	Space Removal	"Hello world C"	"Hello world C"	Pass
3	Title Case	"welcome to linux program"	"Welcome To Linux Program"	Pass
4	Word Frequencies	"The cat and the dog"	the: 2, cat: 1, and: 1, dog: 1	Pass
5	Exit	Choice 6	Program terminates cleanly	Pass

4. Source Code Listing

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>
```

```

#define MAX_PARA_SIZE 1000
#define MAX_WORDS 200
#define MAX_WORD_LEN 100

void readParagraph(char paragraph[], size_t size) {
    if (fgets(paragraph, (int)size, stdin) != NULL) {
        size_t len = strlen(paragraph);
        if (len > 0 && paragraph[len - 1] == '\n') paragraph[len - 1] = '\0';
    }
}

void findLongestAndShortestWords(const char paragraph[], char
longestWord[], char shortestWord[]) {
    char tempPara[MAX_PARA_SIZE];
    strcpy(tempPara, paragraph);
    longestWord[0] = '\0'; shortestWord[0] = '\0';

    char *token = strtok(tempPara, " \\t\\n\\r,.-!?:\\\"'()");
    size_t minLen = 99999, maxLen = 0;

    while (token != NULL) {
        size_t len = strlen(token);
        if (len > maxLen) { maxLen = len; strcpy(longestWord, token); }
        if (len < minLen) { minLen = len; strcpy(shortestWord, token); }
        token = strtok(NULL, " \\t\\n\\r,.-!?:\\\"'()");
    }
}

void removeRepeatedSpaces(const char paragraph[], char
modifiedParagraph[]) {
    int i = 0, j = 0, inSpace = 0;
    while (paragraph[i] != '\0') {
        if (isspace((unsigned char)paragraph[i])) {
            if (!inSpace) { modifiedParagraph[j++] = ' '; inSpace = 1; }
        } else {
            modifiedParagraph[j++] = paragraph[i]; inSpace = 0;
        }
        i++;
    }
}

```

```

    modifiedParagraph[j] = '\0';
}

void convertToTitleCase(const char paragraph[], char
titleCaseParagraph[]) {
    int i = 0, newWord = 1;
    while (paragraph[i] != '\0') {
        char ch = paragraph[i];
        if (isalpha((unsigned char)ch)) {
            titleCaseParagraph[i] = newWord ? (char)toupper((unsigned
char)ch) : (char)tolower((unsigned char)ch);
            newWord = 0;
        } else {
            titleCaseParagraph[i] = ch;
            if (isspace((unsigned char)ch) || ch == '.' || ch == ',' || ch == '!' || ch ==
'?' ) newWord = 1;
        }
        i++;
    }
    titleCaseParagraph[i] = '\0';
}

void toLowercaseWord(char *dest, const char *src) {
    int i = 0;
    while (src[i] != '\0') { dest[i] = (char)tolower((unsigned char)src[i]); i++; }
    dest[i] = '\0';
}

void findWordFrequencies(const char paragraph[]) {
    char tempPara[MAX_PARA_SIZE];
    strcpy(tempPara, paragraph);
    char words[MAX_WORDS][MAX_WORD_LEN];
    int count[MAX_WORDS] = {0}, distinctCount = 0;

    char *token = strtok(tempPara, " \\t\\n\\r,.-!?:\\\"()");
    while (token != NULL) {
        char cleanWord[MAX_WORD_LEN];
        toLowercaseWord(cleanWord, token);
        int found = 0;
        for (int i = 0; i < distinctCount; i++) {

```

```

        if (strcmp(words[i], cleanWord) == 0) { count[i]++; found = 1; break; }
    }
    if (!found && distinctCount < MAX_WORDS) {
        strcpy(words[distinctCount], cleanWord);
        count[distinctCount++] = 1;
    }
    token = strtok(NULL, " \\t\\n\\r,.-!?:\\\"'()");
}

for (int i = 0; i < distinctCount; i++) {
    printf(" - %-15s : %d\\n", words[i], count[i]);
}
}

int main(void) {
    char para1[MAX_PARA_SIZE] = "", para2[MAX_PARA_SIZE] = "";
    int choice;

    printf("Enter Paragraph 1:\\n"); readParagraph(para1, sizeof(para1));
    printf("Enter Paragraph 2:\\n"); readParagraph(para2, sizeof(para2));

    do {
        printf("\\n1. Re-enter Paragraphs\\n2. Longest/Shortest\\n3. Remove
Spaces\\n4. Title Case\\n5. Word Frequencies\\n6. Exit\\nChoice: ");
        if (scanf("%d", &choice) != 1) break;
        getchar();

        switch (choice) {
            case 1:
                printf("Enter Para 1:\\n"); readParagraph(para1, sizeof(para1));
                printf("Enter Para 2:\\n"); readParagraph(para2, sizeof(para2));
                break;
            case 2: {
                char l1[MAX_WORD_LEN], s1[MAX_WORD_LEN],
l2[MAX_WORD_LEN], s2[MAX_WORD_LEN];
                findLongestAndShortestWords(para1, l1, s1);
                findLongestAndShortestWords(para2, l2, s2);
                printf("Para 1 -> Longest: %s, Shortest: %s\\n", l1, s1);
                printf("Para 2 -> Longest: %s, Shortest: %s\\n", l2, s2);
                break;
            }
        }
    } while (choice != 6);
}

```

```

    }
    case 3: {
        char m1[MAX_PARA_SIZE], m2[MAX_PARA_SIZE];
        removeRepeatedSpaces(para1, m1);
removeRepeatedSpaces(para2, m2);
        printf("Para 1 Cleaned: %s\nPara 2 Cleaned: %s\n", m1, m2);
        break;
    }
    case 4: {
        char t1[MAX_PARA_SIZE], t2[MAX_PARA_SIZE];
        convertToTitleCase(para1, t1); convertToTitleCase(para2, t2);
        printf("Para 1 Title Case: %s\nPara 2 Title Case: %s\n", t1, t2);
        break;
    }
    case 5:
        printf("\nPara 1 Frequencies:\n"); findWordFrequencies(para1);
        printf("\nPara 2 Frequencies:\n"); findWordFrequencies(para2);
        break;
    case 6: printf("Exiting...\n"); break;
    }
} while (choice != 6);

return 0;
}

```

5. Execution Screenshots

```
kali@kali:~$ mkdir HD0C_Day41
cd HD0C_Day41

kali@kali:~/HD0C_Day41$ nano day41.c

kali@kali:~/HD0C_Day41$ gcc day41.c -o day41

kali@kali:~/HD0C_Day41$ ./day41
===== READ PARAGRAPHS =====
Enter Paragraph 1:
c programming is very fun. c programming is great!

Enter Paragraph 2:
welcome to kali linux environment. kali linux is powerful.

===== HD0C DAY-41 MENU =====
1. Re-enter Paragraphs
2. Find Longest and Shortest Words
3. Remove Repeated Consecutive Spaces
4. Convert to Title Case
5. Display Distinct Word Frequencies
6. Exit
Enter your choice (1-6): 2

--- Paragraph 1 ---
Longest Word : programming
Shortest Word: c

--- Paragraph 2 ---
Longest Word : environment
Shortest Word: to

===== HD0C DAY-41 MENU =====
1. Re-enter Paragraphs
2. Find Longest and Shortest Words
3. Remove Repeated Consecutive Spaces
4. Convert to Title Case
5. Display Distinct Word Frequencies
6. Exit
Enter your choice (1-6): 4

--- Paragraph 1 (Title Case) ---
C Programming Is Very Fun. C Programming Is Great!

--- Paragraph 2 (Title Case) ---
Welcome To Kali Linux Environment. Kali Linux Is Powerful.

===== HD0C DAY-41 MENU =====
1. Re-enter Paragraphs
2. Find Longest and Shortest Words
3. Remove Repeated Consecutive Spaces
4. Convert to Title Case
5. Display Distinct Word Frequencies
6. Exit
```